SWING OVERVIEW

Objectives

- MVC Architecture
- Swing features
- Swing - Controls
- Common Swing UI Elements:

SWING OVERVIEW

Swing API is set of extensible GUI Components to ease developer's life to create JAVA based Front End/ GUI Applications. It is build upon top of AWT API and acts as replacement of AWT API as it has almost every control corresponding to AWT controls. Swing component follows a Model-View-Controller architecture to fulfill the following criterias.

- A single API is to be sufficient to support multiple look and feel.
- API is to model driven so that highest level API is not required to have the data.
- API is to use the Java Bean model so that Builder Tools and IDE can provide better services to the developers to use it.

MVC Architecture

Swing API architecture follows loosely based MVC architecture in the following manner.

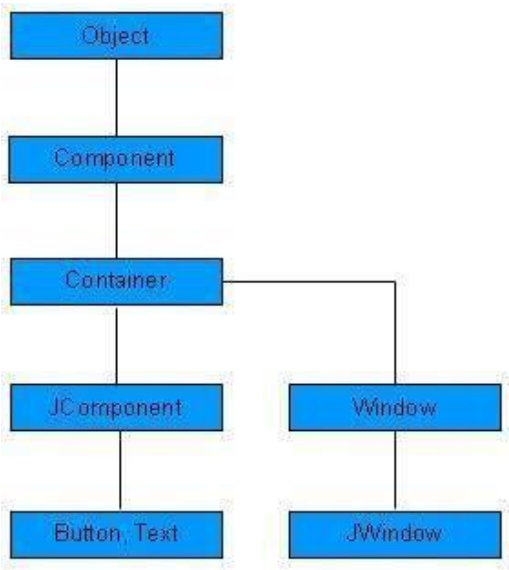
- A Model represents component's data.
- View represents visual representation of the component's data.
- Controller takes the input from the user on the view and reflects the changes in Component's data.
- Swing component have Model as a seperate element and View and Controller part are clubbed in User Interface elements. Using this way, Swing has pluggable look-and-feel architecture.

Swing - Controls

Every user interface considers the following three main aspects:

- **UI elements** : Thes are the core visual elements the user eventually sees and interacts with. GWT provides a huge list of widely used and common elements varying from basic to complex which we will cover in this tutorial.
- **Layouts:** They define how UI elements should be organized on the screen and provide a final look and feel to the GUI (Graphical User Interface). This part will be covered in Layout chapter.
- **Behavior:** These are events which occur when the user interacts with UI elements. This part will be covered in Event
  Handling chapter.

Swing - Controls



Every SWING controls inherit properties from following Component class hiearchy.

Sr. No.	Class & Description

Swing - Controls

1	Component A Component is the abstract base class for the non menu user-interface controls of SWING. Component represents an object with graphical representation
2	Container A Container is a component that can contain other SWING components.
3	JComponent A JComponent is a base class for all swing UI components. In order to use a swing component that inherits from JComponent, component must be in a containment hierarchy whose root is a top-level Swing container.

SWING - JLabel Class

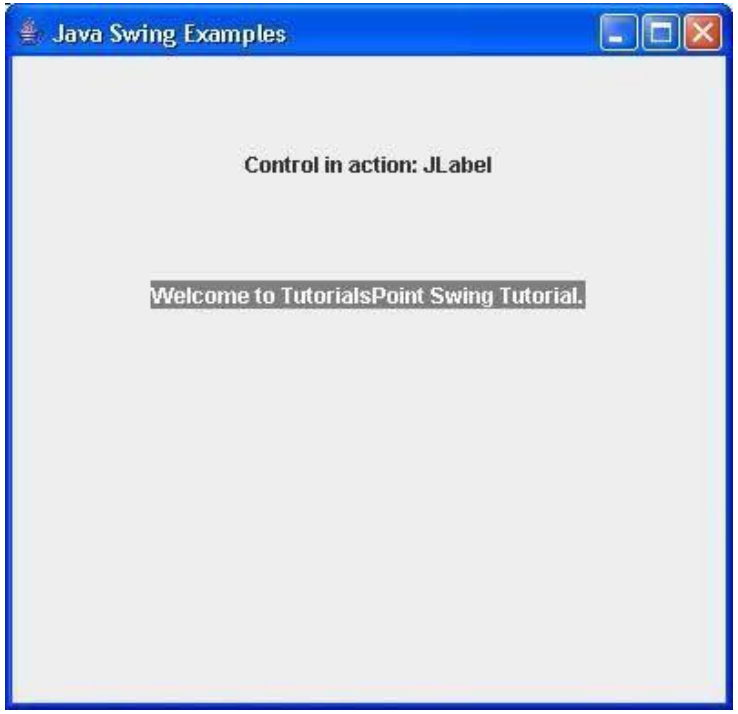
The class **JLabel** can display either text, an image, or both. Label's contents are aligned by setting the vertical and horizontal alignment in its display area. By default, labels are vertically centered in their display area. Text-only labels are leading edge aligned, by default; image-only labels are horizontally centered, by default.

SwingControlDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

SwingControlDemo.showLabelDemo();

output



SWING - JButton Class

The class **JButton** is an implementation of a push button. This component has a label and generates an event when pressed. It can have Image also.

SwingControlDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingControlDemo {
```

SWING - JButton Class

```
  private JFrame mainFrame;
  private JLabel headerLabel;
  private JLabel statusLabel;
  private JPanel controlPanel;

  public SwingControlDemo(){
    prepareGUI();
  }

  public static void main(String[] args){
    SwingControlDemo swingControlDemo = new
SwingControlDemo();
    swingControlDemo.showButtonDemo();
  }
```

SWING - JButton Class

```
private void prepareGUI(){
 mainFrame = new JFrame("Java Swing Examples");
 mainFrame.setSize(400,400);
 mainFrame.setLayout(new GridLayout(3, 1));
 mainFrame.addWindowListener(new WindowAdapter() {
   public void windowClosing(WindowEvent windowEvent){
        System.exit(0);
   }
 });
 headerLabel = new JLabel("", JLabel.CENTER);
 statusLabel = new JLabel("",JLabel.CENTER);

 statusLabel.setSize(350,100);

 controlPanel = new JPanel();
```

SWING - JButton Class

```
    controlPanel.setLayout(new FlowLayout());

    mainFrame.add(headerLabel);
    mainFrame.add(controlPanel);
    mainFrame.add(statusLabel);
    mainFrame.setVisible(true);
  }

  private static ImageIcon createImageIcon(String path,  String
    description) {
    java.net.URL imgURL =
SwingControlDemo.class.getResource(path);
    if (imgURL != null) {
      return new ImageIcon(imgURL, description);
    } else {
```

SWING - JButton Class

```
        System.err.println("Couldn't find file: " + path);
        return null;
    }
  }

  private void showButtonDemo(){

    headerLabel.setText("Control in action: Button");


    //resources folder should be inside SWING folder.
    ImageIcon icon =
createImageIcon("/resources/java_icon.png","Java");

    JButton okButton = new JButton("OK");
    JButton javaButton = new JButton("Submit", icon);
```

SWING - JButton Class

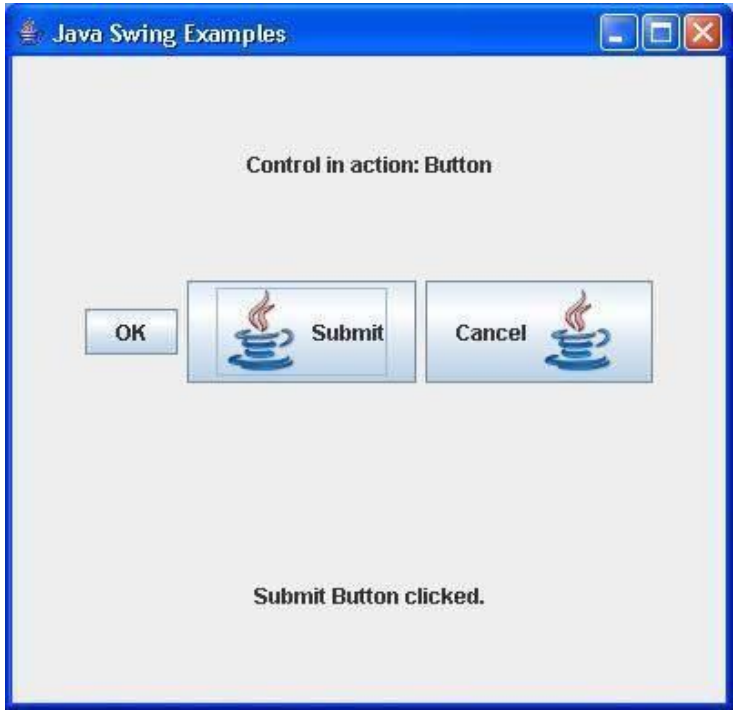
```
    JButton cancelButton = new JButton("Cancel", icon);

cancelButton.setHorizontalTextPosition(SwingConstants.LEFT)

    ;  okButton.addActionListener(new ActionListener() {
       public void actionPerformed(ActionEvent e) {
          statusLabel.setText("Ok Button clicked.");
       }
    });

    javaButton.addActionListener(new ActionListener() {
       public void actionPerformed(ActionEvent e) {
          statusLabel.setText("Submit Button clicked.");
       }
    });
```

SWING - JButton Class

```
    cancelButton.addActionListener(new ActionListener() {
      public void actionPerformed(ActionEvent e) {
        statusLabel.setText("Cancel Button clicked.");
      }
    });

    controlPanel.add(okButton);
    controlPanel.add(javaButton);
    controlPanel.add(cancelButton);

    mainFrame.setVisible(true);
  }
}
```

SWING - JButton Class

output



SWING - JCheckBox Class

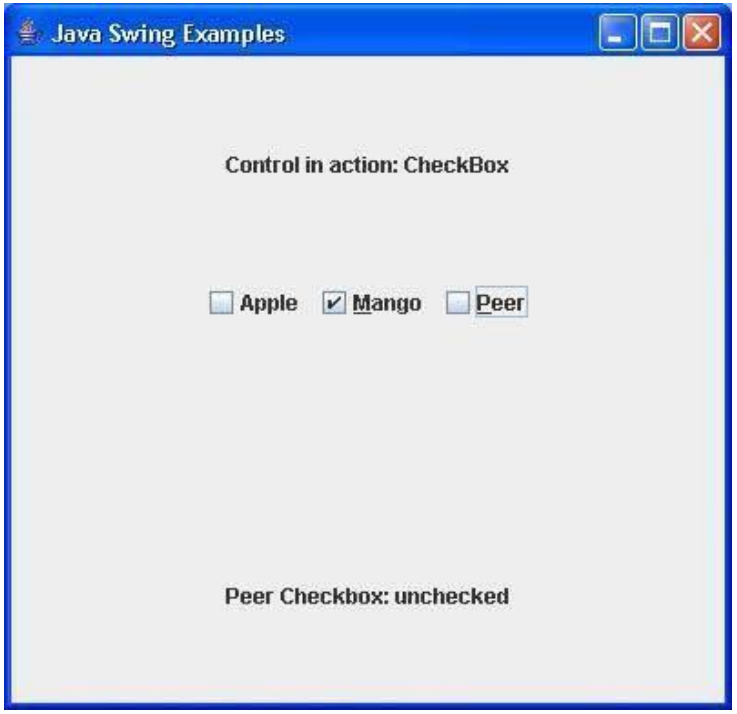
The class **JCheckBox** is an implementation of a check box - an item that can be selected or deselected, and which displays its state to the user.

SwingControlDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingControlDemo {

   private JFrame mainFrame;
```

SWING - JCheckBox Class

output



SWING - JRadioButton Class

The class **JRadioButton** is an implementation of a radio button - an item that can be selected or deselected, and which displays its state to the user.

SwingControlDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingControlDemo {

   private JFrame mainFrame;
   private JLabel headerLabel;
```

SWING - JRadioButton Class

output

SWING - JTextField Class

The class **JTextField** is a component which allows the editing of a single line of text.

SwingControlDemo.java

```
package com.tutorialspoint.gui;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingControlDemo {

   private JFrame mainFrame;
```

SWING - JTextField Class

```
  private JLabel headerLabel;
  private JLabel statusLabel;
  private JPanel controlPanel;

  public SwingControlDemo(){
    prepareGUI();
  }

  public static void main(String[] args){
    SwingControlDemo swingControlDemo = new
SwingControlDemo();
    swingControlDemo.showTextFieldDemo();
  }

  private void prepareGUI(){
```

SWING - JTextField Class

```
mainFrame = new JFrame("Java Swing Examples");
mainFrame.setSize(400,400);  mainFrame.setLayout(new
GridLayout(3, 1));
mainFrame.addWindowListener(new WindowAdapter() {  public
   void windowClosing(WindowEvent windowEvent){
      System.exit(0);
   }
});
headerLabel = new JLabel("", JLabel.CENTER);
statusLabel = new JLabel("",JLabel.CENTER);

statusLabel.setSize(350,100);

controlPanel = new JPanel();
controlPanel.setLayout(new FlowLayout());
```

SWING - JTextField Class

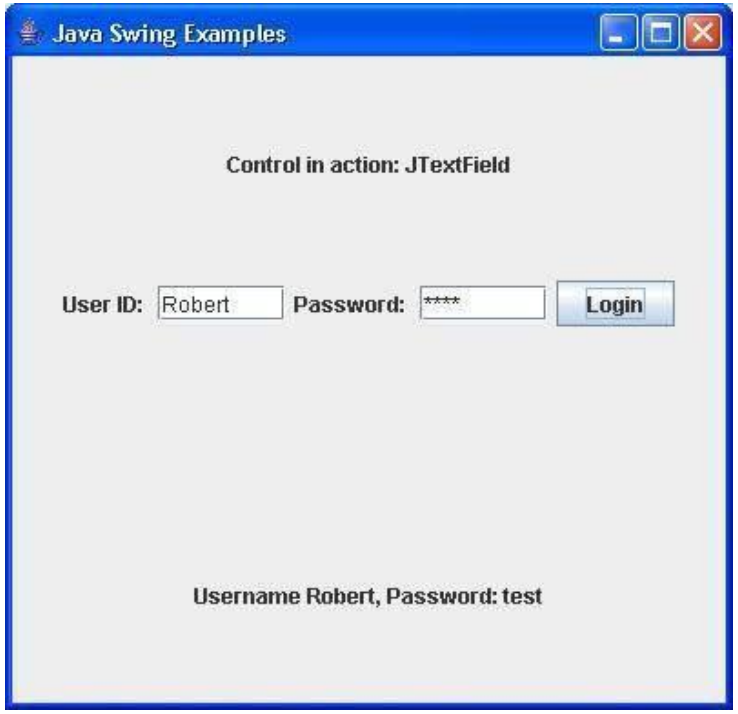
```
mainFrame.add(headerLabel);
mainFrame.add(controlPanel);
mainFrame.add(statusLabel);
mainFrame.setVisible(true);
   }

   private void showTextFieldDemo(){
      headerLabel.setText("Control in action: JTextField");

      JLabel  namelabel= new JLabel("User ID: ", JLabel.RIGHT);
      JLabel  passwordLabel = new JLabel("Password: ",
JLabel.CENTER);
      final JTextField userText = new JTextField(6);
      final JPasswordField passwordText = new JPasswordField(6);
```

SWING - JTextField Class

```
JButton loginButton = new JButton("Login");
loginButton.addActionListener(new ActionListener() {
  public void actionPerformed(ActionEvent e) {  String data =
    "Username " + userText.getText();  data += ", Password: "
    + new String(passwordText.getPassword());
    statusLabel.setText(data);
  }
});

controlPanel.add(namelabel);
controlPanel.add(userText);
controlPanel.add(passwordLabel);
controlPanel.add(passwordText);
```

SWING - JTextField Class

```
        controlPanel.add(loginButton);
        mainFrame.setVisible(true);
    }
}
```

SWING - JTextField Class

output